Context-Adaptive Multi-Critic Controller

using Deep Learning-Based Cost Weighting in Autonomous Navigation

Pravin Oli | Erasmus Mundus Joint Master in Intelligent Field Robotic Systems (IFRoS)

Supervisor: **Prof. Z. Istenes** (ELTE) | Industrial: **R. Tommasi** & **Dr. D. Horváth** (European Knowledge Centre Ltd.)

LinkedIn (author) | YouTube (video) | GitHub (repo)

1. Motivation & Problem

A mobile robot follows a global route with a **local controller** that decides its behaviour cycle-by-cycle.

- **Classical samplers** (DWB, MPPI): a *weighted sum of interpretable costs*, safe and transparent, but the weights are *fixed* and cannot suit every context.
- **End-to-end learned** controllers adapt but are *opaque*; safety is only a learned regularity, not a guarantee.

Gap: adapt to context *every cycle* while keeping the hard safety of the classical design.

Objective. Learn *only* the per-cycle cost weighting; leave trajectory generation and safety to the classical planner.

Research question. Does this beat both fixed-weight classical planners and an end-to-end learned baseline on a multi-criteria aggregate, without sacrificing hard safety?

2. Methodology

A — Overview: Safety by Decomposition

CA-MCW. A context-conditioned network emits per-cycle multipliers $\mathbf{m}_t$ that rescale the configured critic weights, $w_c^{\text{eff}} = m_{c,t}\, w_c^{\circ}$ $(m_{c,t} \geq 0.05)$; generation, feasibility and safety *stay inside the classical planner* (Fig. 1).

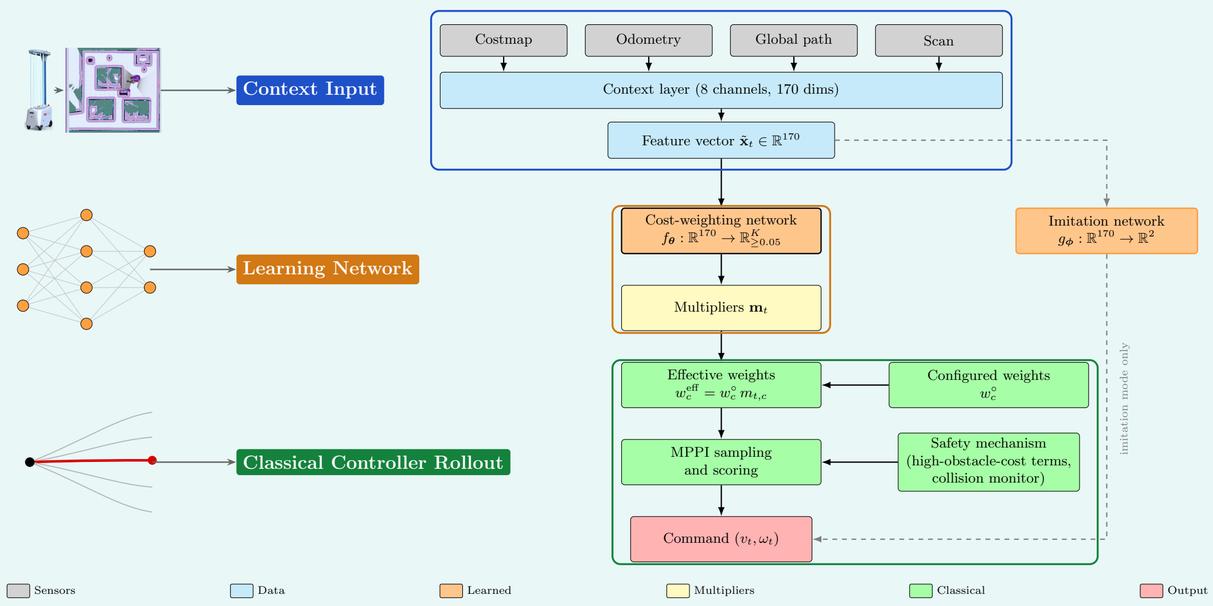
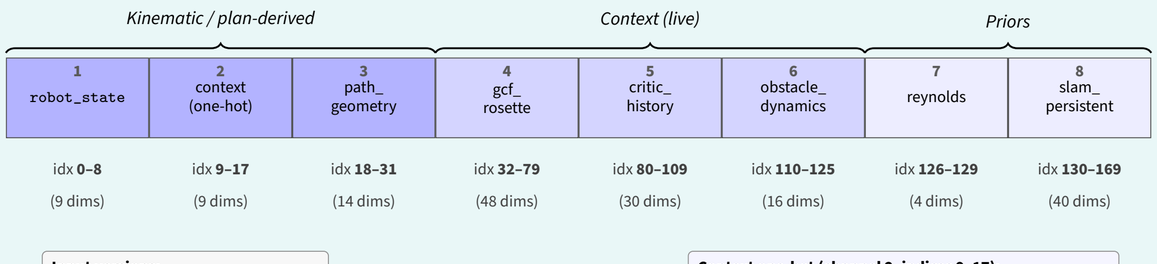


Fig. 1 Pipeline grouped into context, learned network, and classical rollout.

B — Context Input (170-D Feature Vector)

A single 170-D vector of eight named channels (three provenance families) conditions the network each cycle.



Input versions:
v1 = channels 1–6 (126 dims)
v2 = v1 + channel 7 (130 dims)
v3 = v2 + channel 8 (170 dims, headline)

Context one-hot (channel 2, indices 9–17):

9: open	12: junction	15–17: reserved (zero)
10: corridor	13: cluttered	
11: doorway	14: approaching-goal	

Fig. 2 Index-range layout of the v3 context vector $\mathbf{x}_t$.

C — Cost-Weighting Network

A channel-wise frontend feeds a (256, 128, 64) fusion stack and a paradigm-specific head: $64 \to 11$ + softplus for the multiplier paradigms, $64 \to 2$ for the imitation baseline.

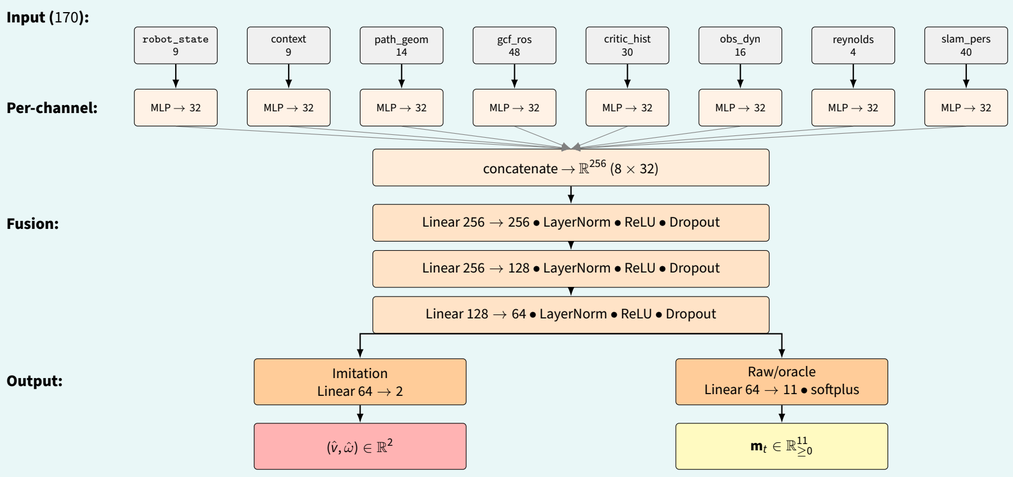


Fig. 3 End-to-end structure of the learned controllers.

D — Weighted MPPI Rollout

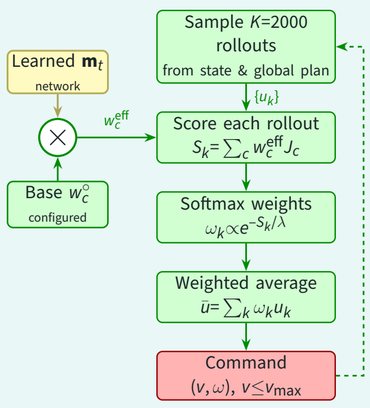


Fig. 4 Weighted MPPI control cycle; repeats every step.

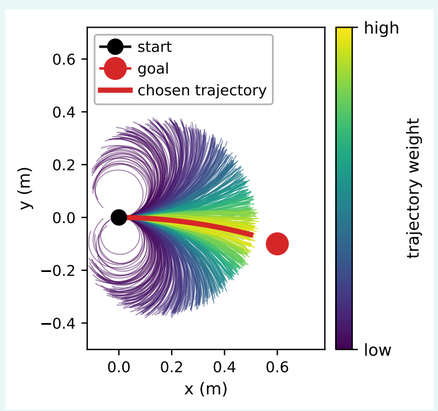


Fig. 5 K=2000 rollouts over a 2.0 s horizon (illustrative).

E — Control Algorithm & Critic Taxonomy

Algorithm 1 — CA-MCW control cycle

1: $\mathbf{x}_t \leftarrow$ `assembleFeatures`(costmap, odom, path, scan)
2: $\mathbf{m}_t \leftarrow$ `weightProvider`$(\mathbf{x}_t)$
3: **for** cost term $c \in \{1, \ldots, K\}$ **do**
4: $\quad w_c^{\text{eff}} \leftarrow w_c^{\circ} \cdot m_{t,c}$
5: **end for**
6: $\tau^{\star} \leftarrow$ `MPPI`$\left(\{w_c^{\text{eff}}\}_{c=1}^{K}\right)$
7: $(v_t, \omega_t) \leftarrow \tau^{\star}[0]$
8: **return** (v_t, ω_t)

The multiplier enters *only* the cost-weight product; feasibility and command extraction stay with the planner.

Reynolds-grouped critics

Ten cost terms reorganised on Reynolds' flocking primitives, adapted to single-robot navigation:

Primitive	Role
Separation	clear of obstacles / walls
Alignment	match heading to plan
Cohesion	stay on the global path
Goal-seeking	progress to the goal

3. Controlled Experimental Design

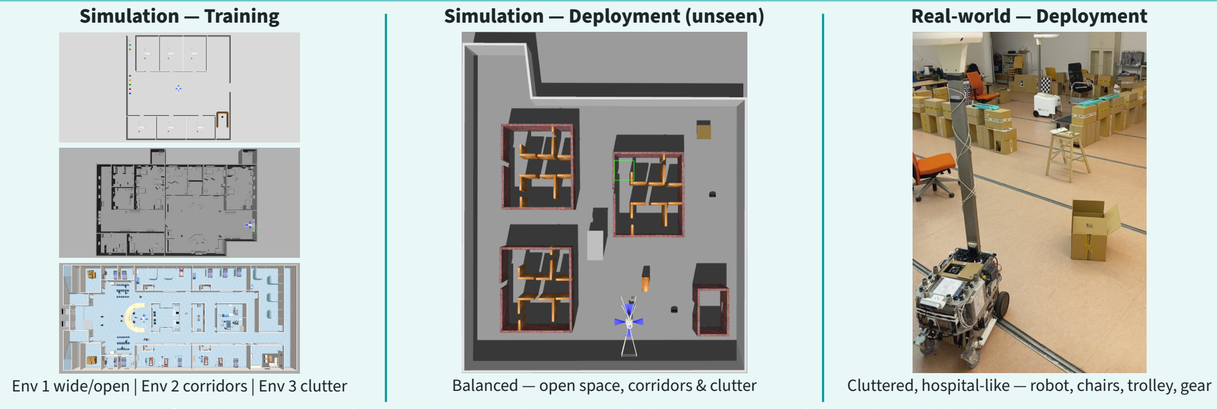


Fig. 6 Environments: three training (sim), the unseen deployment (sim), and the real-world deployment.

4. Training Convergence

One representative model per paradigm — train (solid) vs. validation (dashed); early stopping fires before overfitting.

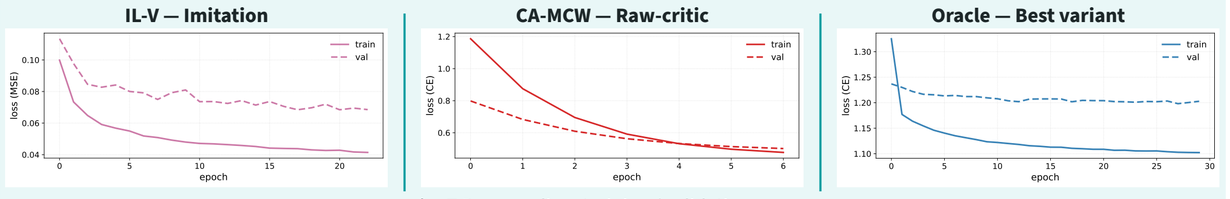


Fig. 7 Per-paradigm training / validation curves.

5. Results

A — Learned Adaptation Signature

Each cost's weight varies *systematically* with obstacle proximity: a **context-conditioned schedule**, not a constant vector. Near obstacles the controller up-weights the obstacle and corridor costs and down-weights goal pull.

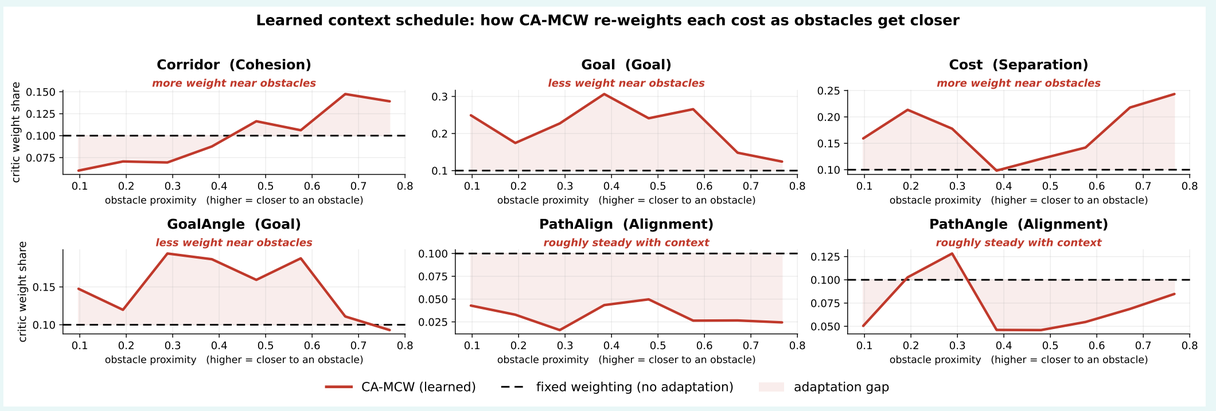


Fig. 8 Learned per-cost weighting vs. obstacle proximity (six most context-sensitive costs).

Adaptation signature: a context-driven cost balance that no fixed weighting can reproduce.

B — Path Adherence

Cross-track error against each controller's own active plan: the five completing controllers track tightly (≈ 0.02 m).

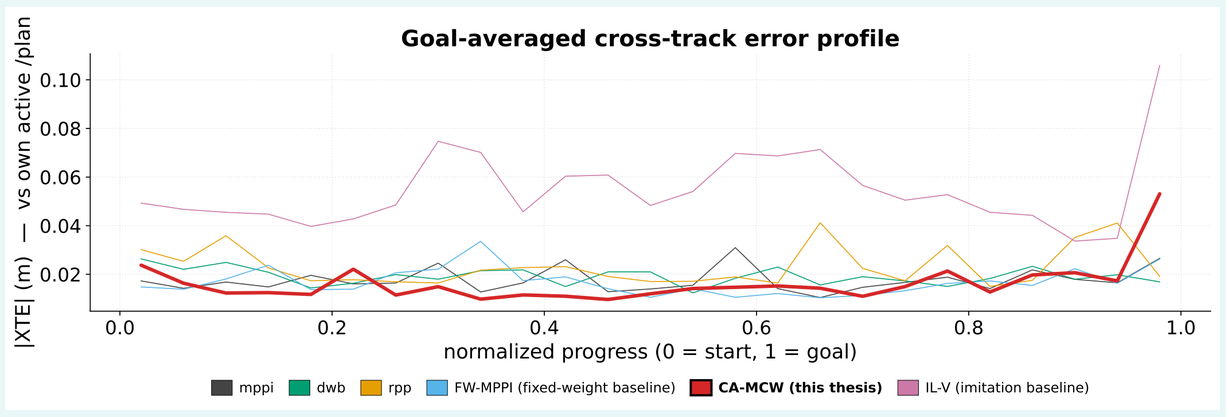


Fig. 9 Goal-averaged cross-track error vs. own active plan over normalised progress.

C — Tour & Overall Ranking (CA-MCW Ranks First)

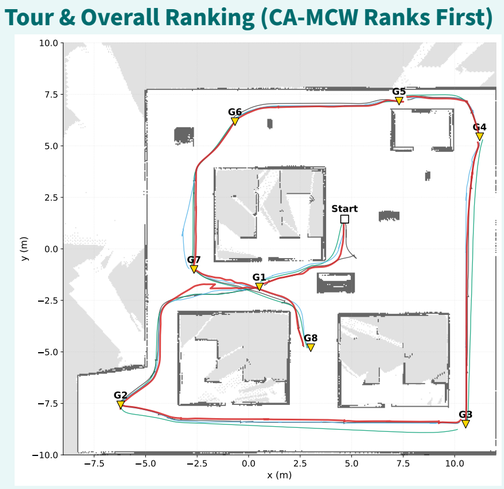


Fig. 10 Eight-goal evaluation tour (fully-successful controllers).

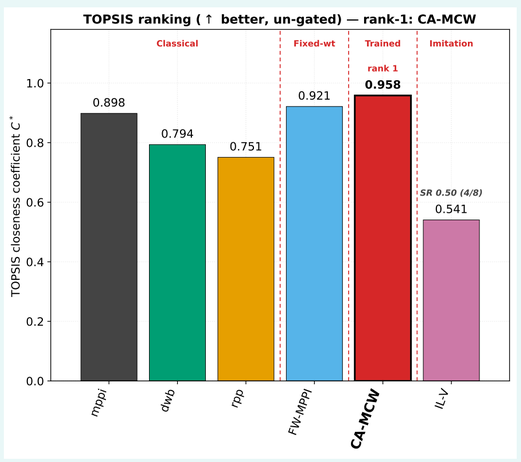


Fig. 11 TOPSIS closeness $C^{\star}$ (higher better).

mppi | dwb | rpp | FW-MPPI (fixed-weight baseline) | **CA-MCW (this thesis)** | IL-V (imitation baseline)

CA-MCW (proposed): 8/8 goals, **zero collisions**; best worst-case clearance, smoothness and goal accuracy; concedes only raw speed.
FW-MPPI: fixed-weight MPPI; 8/8, but one weighting for all contexts.
mppi, **dwb**: classical baselines; 8/8, weaker path quality, no adaptation.
rpp: regulated pure pursuit; misses a goal (7/8).
IL-V: end-to-end imitation; only **4/8** goals and the *only* collider.

Controller	$C^{\star}$
CA-MCW	**0.96**
FW-MPPI	0.92
mppi	0.90
dwb	0.79
rpp	0.75
IL-V	0.54

6. Conclusion & Future Work

Conclusion. Splitting the controller into a *learned* per-cycle cost weighting and a *classical* safety-bearing MPPI planner lets CA-MCW adapt to context without surrendering the hard guarantees of sampling-based control. It tops every classical baseline, the fixed-weight reference and every oracle variant on the multi-criteria aggregate, and has been **successfully deployed on the physical robot**, not only in simulation.

- **Dynamic scenes:** moving obstacles and crowds beyond the static tour.
- **Joint learning:** train base weights w_c° and multipliers $\mathbf{m}_t$ end-to-end rather than rescaling a fixed configuration.
- **Broader real-world trials:** longer missions and a quantitative sim-to-real gap study.
- **Scaling:** larger critic banks and richer context channels.

Separating *what is learned* (weighting) from *what stays classical* (safety) yields a controller that is both context-adaptive *and* safe by construction; the best-balanced of all 23 evaluated.